

Lập trình Shell (2)

Bài 1. Viết script in ra các hình sau

Hình chữ nhật: thông số dài, rộng là các tham số của script.

Các hình còn lại: chỉ cần 1 kích thước là đủ để vẽ.

```
chu_nhat      tam_giac_duoi  tam_giac_tren  tam_giac_so
* * * * *    *              * * * *      1
* * * * *    * *           * * *        2 3
* * * * *    * * *        * *          4 5 6
* * * * *    * * * *      *            7 8 9 10
```

Done

Biến tự động (1)

- Là các biến do hệ thống tự động tạo ra, liên quan đến tham số dòng lệnh của chương trình shell

Tên biến	Ý nghĩa
\$0	Chứa tên lệnh
\$1,...,\$9	Chứa giá trị các tham số dòng lệnh, từ trái sang phải tương đương với từ bé đến lớn
\$#	Chứa tổng số các tham số dòng lệnh không tính biến \$0
\$*	Toàn bộ các tham số dòng lệnh được ghép thành 1 chuỗi
\$?	chứa giá trị kết quả trả lại của câu lệnh trước

Hình 1: Biến tự động (1)

Bài 2a. Viết chương trình tính tổng các tham số nhập vào từ dòng lệnh

Ví dụ: \$ tong 3 2 4 5 7

In ra kết quả: Tong cac so la: 21

```
sum=0
for num in "$@"; do
    sum=$((sum + num))
done
echo "Tong cac so la: $sum"
```

Bài 2b. Hãy lập chương trình cho phép người dùng tìm số lớn nhất trong số các tham số dòng lệnh

Ví dụ: \$ lon_nhat 3 2 4 5 7

In ra kết quả: So lon nhat la: 7

```
max=$1
for num in "$@"; do
    if [ $num -gt $max ]; then
        max=$num
    fi
done
echo "So lon nhat la: $max"
```

Bài 3. Hãy lập chương trình in ra số ngày của một tháng trong năm

Ví dụ: \$ songay 3 2011

In ra kết quả: Tháng 3 của năm 2011 có 31 ngày

Ví dụ: \$ songay 2 2000

In ra kết quả: Tháng 2 của năm 2000 có 29 ngày

Gợi ý:

- Dùng ((nam % 4 == 0)) nếu dùng (()).
- Dùng [\$((nam % 4)) -eq 0] nếu dùng [].

Toán học:

```
#!/bin/bash
thang=$1
nam=$2
if [ $# -ne 2 ]; then
    echo "Vui long nhap du tham so thang-nam"
    exit 1
fi
if ! (( thang + 0 )) 2>/dev/null || ! (( nam + 0 )) 2>/dev/null; then
    echo "Vui long nhap dung dang so nguyen"
    exit 1
fi
if [ $thang -lt 1 ] || [ $thang -gt 12 ]; then
    echo "Vui long nhap thang trong khoang 1-12"
    exit 1
fi
```

```

case $thang in
    1|3|5|7|8|10|12) songay=31;;
    4|6|9|11) songay=30;;
    2)
        if (( nam % 4 == 0 )); then
            songay=29
        else
            songay=28
        fi
        ;;
    *)
        echo "Thang khong hop le"
        exit 1
        ;;
esac
echo "Thang $thang nam $nam co $songay ngay"

```

Bài 4. Viết chương trình tính giai thừa của n theo 2 cách: dùng vòng lặp for, dùng vòng lặp while

Bài 5a. Hãy lập chương trình cho phép nhập vào một đường dẫn tuyệt đối

Đưa ra thông báo nếu nhập đường dẫn sai, nếu nhập đúng thì đưa ra các quyền của người sử dụng đối với đường dẫn đó.

Bài 5b. Với 10 file/thư mục (bất kỳ) của /etc, em hãy hiển thị thông tin chi tiết về chúng theo dạng “thân thiện” hơn

Ví dụ:

```

adduser.conf
    Day la mot file
    Ban co quyen doc adduser.conf
    Ban khong co quyen ghi trong adduser.conf
    Ban khong so huu adduser.conf
    Ban khong phai thanh vien cua nhom so huu adduser.conf

```

```

#!/bin/bash
# Chon ngau nhien 10 file/thu muc tu /etc
files=$(ls /etc | shuf -n 10)

for file in "${files[@]"; do
    filepath="/etc/$file"

    echo "$file"

    if [ -f "$filepath" ]; then
        echo -e "\tDay la mot file"
    fi
done

```

```

elif [ -d "$filepath" ]; then
    echo -e "\tDay la mot thu muc"
else
    echo -e "\tDay la mot loai tep khac"
fi

if [ -r "$filepath" ]; then
    echo -e "\tBan co quyen doc $file"
else
    echo -e "\tBan khong co quyen doc $file"
fi

if [ -w "$filepath" ]; then
    echo -e "\tBan co quyen ghi trong $file"
else
    echo -e "\tBan khong co quyen ghi trong $file"
fi

if [ "$(stat -c %u "$filepath")" -eq "$UID" ]; then
    echo -e "\tBan so huu $file"
else
    echo -e "\tBan khong so huu $file"
fi

if id -nG | grep -qw "$(stat -c %G "$filepath")"; then
    echo -e "\tBan la thanh vien cua nhom so huu $file"
else
    echo -e "\tBan khong phai thanh vien cua nhom so huu $file"
fi

echo
done

```

Bài 6. Copy file có quyền đọc từ /etc về ~/myetc và chuyển thành file dạng txt

Ví dụ: với file `adduser.conf` thì chuyển thành `~/myetc/adduser.conf.txt`

Bài 7. Hiển thị thông tin về các đường dẫn trong \$PATH theo dạng thân thiện hơn

Thực hiện chương trình để xem kết quả. Chuyển sang tài khoản root. Thực hiện lại chương trình và so sánh kết quả.

Bài 8. Chương trình kiểm tra mật khẩu

Trong chương trình của em có một biến lưu giá trị password (là một chuỗi bất kỳ nào đó). Chương trình cho phép nhập một giá trị gọi là mật khẩu. Nếu mật khẩu trùng password thì thông báo “Chúc mừng bạn”, nếu mật khẩu nhập sai thì yêu cầu nhập lại, nhưng không được

nhập quá 5 lần, nếu nhập quá thì thông báo ra là “Khoa tai khoan”. Gợi ý: dùng vòng lặp while hoặc until kết hợp với biến đếm vòng lặp.

Bài 9. Sử dụng chương trình gợi ý để viết chương trình mới

a. Chương trình đếm số dòng của một tập tin (tương tự `$ wc -l file`)

Minh họa các cấu trúc if then fi, while do done, và cách sử dụng [], \$(()).

Tập tin demdong.sh:

```
#!/bin/sh
echo "Chương trình đếm số dòng của tập tin $1"
n=0
while read line
do
n=$((n + 1))
done < $1
echo "Số dòng của tập tin $1 là : $n"
exit 0
```

Chạy chương trình:

```
chmod a+x demdong.sh
./demdong bai1.txt
```

b. Chương trình đếm số từ của một tập tin (tương tự `$ wc -w file`)

Minh họa các cấu trúc for do done, while do done.

Tập tin demtu.sh:

```
#!/bin/sh
echo "Chương trình đếm số từ của tập tin $1"
n=0
while read line
do
    for wd in $line
    do
        n=$((n + 1))
    done
done < $1
echo "Tổng số từ của tập tin $1 là : $n"
exit 0
```

Chạy chương trình:

```
chmod a+x demtu.sh
./demptu bai1.txt
```

Hỏi: Từ 2 chương trình gợi ý a & b ở trên, em hãy viết chương trình mywc thực hiện đếm số dòng, số từ trong tập tin, với các cách dùng như sau:

`$ mywc file` – đưa ra số dòng, số từ của tập tin đó

Tham số:

- -l: đưa ra số dòng của file
- -w: đưa ra số từ của file

Chú ý: Không được sử dụng `wc` ☹

Sửa chương trình sau để có thể chạy đúng

Bài 10. Chương trình tìm dòng có độ dài lớn nhất trong một tập tin

Minh họa các cấu trúc `if then fi`, `while do done`.

Tập tin `dongmax.sh`:

```
#!/bin/sh
echo "Chương trình tìm dòng dài nhất trong tập tin $1"
{
n=0
max=0
dong=""
while read line
do
n='expr length "$line"'
if [ $n -gt $max ]
then
dong="$line"
max=$n
fi
done
echo "Dòng trong tập tin $1 có độ dài max = $max là : $dong"
}<$1
exit 0
```

Chạy chương trình:

```
chmod a+x dongmax.sh
./dongmax.sh bai1.txt
```

Sửa chương trình sau để có thể chạy đúng

Bài 11. Chương trình tìm một xâu trong một tập tin

Minh họa các cấu trúc `if then fi`, `while do done`.

Tập tin `timxau.sh`:

```
#!/bin/sh
echo "Chương trình tìm xâu $1 trong tập tin $2"
{
wordlen='expr length "$1"' # Độ dài từ cần tìm
while read textline
do
textlen='expr length "$textline"' # Độ dài của dòng vừa đọc
end=$((textlen - wordlen + 1))
index=1
while [ $index -le $end ]
```

```
do
temp='expr substr "$textline" $index $wordlen'
if [ "$temp" = "$1" ]
then
echo "Tim thay $1 tai dong $textline"
break
fi
index=$(( $index + 1 ))
done
done
}<$2
exit 0
```

Chạy chương trình:

```
chmod a+x timxau.sh
./timxau.sh abc bai1.txt
```

Các câu lệnh lập trình Shell cơ bản

1. Quản lý tập tin và thư mục

- `ls` → Liệt kê danh sách tập tin, thư mục.
- `cd <thư_mục>` → Di chuyển đến thư mục.
- `pwd` → Hiển thị đường dẫn thư mục hiện tại.
- `mkdir <tên_thư_mục>` → Tạo thư mục mới.
- `rm <tập_tin>` → Xóa tập tin.
- `rm -r <thư_mục>` → Xóa thư mục và toàn bộ nội dung bên trong.
- `cp <tập_tin_nguồn> <tập_tin_đích>` → Sao chép tập tin.
- `mv <tập_tin_cũ> <tập_tin_mới>` → Đổi tên hoặc di chuyển tập tin.
- `touch <tập_tin>` → Tạo tập tin rỗng mới.

2. Xử lý nội dung tập tin

- `cat <tập_tin>` → Hiển thị nội dung tập tin.
- `less <tập_tin>` → Xem nội dung tập tin (cuộn lên xuống).
- `head -n <số_dòng> <tập_tin>` → Hiển thị dòng đầu của tập tin.
- `tail -n <số_dòng> <tập_tin>` → Hiển thị dòng cuối của tập tin.
- `grep "chuỗi"<tập_tin>` → Tìm kiếm chuỗi trong tập tin.
- `sed 's/chuỗi_cũ/chuỗi_mới/g' <tập_tin>` → Thay thế chuỗi trong tập tin.
- `awk -F"," '{print $1, $3}' <tập_tin>` → Xử lý dữ liệu cột.

3. Biến và tham số trong Shell

- `tên_biến="giá trị"` → Gán giá trị cho biến.
- `echo $tên_biến` → In giá trị biến ra màn hình.
- `read tên_biến` → Nhập dữ liệu từ bàn phím.
- `export tên_biến="giá trị"` → Xuất biến môi trường.
- `set` → Liệt kê tất cả biến môi trường.
- `env` → Hiển thị biến môi trường hiện tại.

4. Cấu trúc điều khiển trong Shell

Câu lệnh rẽ nhánh if:

```
if [ $a -gt $b ]; then
    echo "$a lon hon $b"
elif [ $a -lt $b ]; then
    echo "$a nho hon $b"
else
    echo "$a bang $b"
fi
```

Vòng lặp for:

```
for i in {1..5}; do
    echo "Lan $i"
done
```

Vòng lặp while:

```
count=1
while [ $count -le 5 ]; do
    echo "Lan $count"
    count=$((count + 1))
done
```

5. Quản lý tiến trình

- `ps aux` → Xem danh sách tiến trình đang chạy.
- `top` hoặc `htop` → Giám sát tiến trình.
- `kill <PID>` → Dừng tiến trình theo ID.
- `killall <tên_lệnh>` → Dừng tất cả tiến trình có tên lệnh.
- `nohup <lệnh> &` → Chạy tiến trình nền không bị gián đoạn khi đóng terminal.

6. Điều hướng đầu vào/ra

- > → Ghi đè đầu ra vào tập tin.

```
echo "Hello" > file.txt
```

- >> → Ghi thêm đầu ra vào tập tin.

```
echo "Them dong moi" >> file.txt
```

- < → Đọc đầu vào từ tập tin.

```
sort < file.txt
```

- | → Truyền đầu ra của lệnh này vào lệnh khác.

```
cat file.txt | grep "tu_khoa"
```

7. Kiểm tra thuộc tính tập tin

```
if [ -f file.txt ]; then  
    echo "Tap tin ton tai"  
fi
```

Các toán tử kiểm tra:

- -f → Kiểm tra tập tin tồn tại.
- -d → Kiểm tra thư mục tồn tại.
- -r → Kiểm tra quyền đọc.
- -w → Kiểm tra quyền ghi.
- -x → Kiểm tra quyền thực thi.

8. Viết script Shell cơ bản

Lưu script vào `script.sh`, rồi cấp quyền chạy:

```
chmod +x script.sh  
./script.sh
```

Ví dụ script đơn giản:

```
#!/bin/bash  
echo "Nhap ten cua ban:"  
read name  
echo "Chao $name!"
```

1. Biến và tham số dòng lệnh

Khai báo biến:

```
myvar="Hello"  
echo $myvar
```

Tham số dòng lệnh:

```
echo "Tham so thu nhat: $1"  
echo "Tong so tham so: $#"
```

Kiểm tra tham số dòng lệnh:

```
if [ $# -eq 0 ]; then  
    echo "Khong co tham so nao."  
fi
```

Lệnh shift để xử lý nhiều tham số:

```
while [ $# -gt 0 ]; do  
    echo "Tham so hien tai: $1"  
    shift  
done
```

2. Cấu trúc điều khiển

Câu lệnh if:

```
if [ $a -gt $b ]; then  
    echo "$a lon hon $b"  
elif [ $a -lt $b ]; then  
    echo "$a nho hon $b"  
else  
    echo "$a bang $b"  
fi
```

Vòng lặp for:

```
for i in {1..5}; do  
    echo "Lan $i"  
done
```

Vòng lặp while:

```
count=1  
while [ $count -le 5 ]; do  
    echo "Lan $count"  
    count=$((count + 1))  
done
```

Vòng lặp until:

```
until [ $count -gt 5 ]; do  
    echo "Lan $count"  
    count=$((count + 1))  
done
```

Cấu trúc case:

```
case "$1" in
    sang) echo "Good morning!" ;;
    chieu) echo "Good afternoon!" ;;
    toi) echo "Good night!" ;;
    *) echo "Hello!" ;;
esac
```

3. Xử lý chuỗi và tập tin

Tìm kiếm chuỗi trong tập tin:

```
grep "tu_khoa" file.txt
```

Đếm số dòng chứa chuỗi:

```
grep -c "tu_khoa" file.txt
```

Thay thế ký tự trong chuỗi:

```
echo "Hello::World" | tr "::" ","
```

Cắt chuỗi bằng cut:

```
echo "ngochan@yahoo.com.vn" | cut -d'@' -f1 # Lay ten tai khoan
echo "ngochan@yahoo.com.vn" | cut -d'@' -f2 # Lay ten mien
```

Tách tên tập tin và đường dẫn:

```
filename=$(basename "/path/to/file.txt")
dirname=$(dirname "/path/to/file.txt")
```

4. Xử lý tập tin và thư mục

Kiểm tra tập tin tồn tại:

```
if [ -f "file.txt" ]; then
    echo "Tập tin ton tai"
fi
```

Kiểm tra quyền tập tin:

```
if [ -r "file.txt" ]; then
    echo "Ban co quyen doc file"
fi
```

Đổi phần mở rộng của tập tin:

```
for file in *.jpg; do
    mv "$file" "${file%.jpg}.png"
done
```

Copy file:

```
cp /etc/passwd ~/passwd_copy
```

Lấy dòng cụ thể từ tập tin:

```
sed -n '5p' file.txt # In dong thu 5
```

Tìm kiếm file trong thư mục:

```
find . -name "*chuoitimkiem"
```

5. Xử lý số và toán học

Tính tổng các tham số dòng lệnh:

```
sum=0
for num in "$@"; do
    sum=$((sum + num))
done
echo "Tong cac so la: $sum"
```

Tìm số lớn nhất từ tham số dòng lệnh:

```
max=$1
for num in "$@"; do
    if [ $num -gt $max ]; then
        max=$num
    fi
done
echo "So lon nhat la: $max"
```

Tính giai thừa với for:

```
fact=1
for (( i=1; i<=$1; i++ )); do
    fact=$((fact * i))
done
echo "Giai thua cua $1 la $fact"
```

Tính giai thừa với while:

```
fact=1
num=$1
while [ $num -gt 0 ]; do
    fact=$((fact * num))
    num=$((num - 1))
done
echo "Giai thua cua $1 la $fact"
```

Tính số ngày trong tháng:

```
cal $1 $2 | awk 'NF {DAYS = $NF}; END {print DAYS}'
```

6. Quản lý hệ thống và tiến trình

Xem tiến trình đang chạy:

```
ps aux | grep process_name
```

Dừng tiến trình:

```
kill <PID>
```

Lưu biến môi trường PATH và khôi phục lại:

```
echo $PATH > oldpath.txt
export PATH=""
source oldpath.txt
```

Thay đổi biến môi trường PS1:

```
export PS1="MyShell> "
```

Thay đổi biến môi trường PS2:

```
bash
export PS2="Tiep tục nhập lệnh> "
```

7. Xử lý email và dữ liệu có cấu trúc

Tách email thành tên và nhà cung cấp:

```
email="ngochan@yahoo.com.vn"
user=$(echo $email | cut -d'@' -f1)
domain=$(echo $email | cut -d'@' -f2 | cut -d'.' -f1)
ext=$(echo $email | cut -d'@' -f2 | cut -d'.' -f2-)
echo "User: $user, Domain: $domain, Extension: $ext"
```

Thống kê số lượng email theo nhà cung cấp:

```
cut -d'@' -f2 email_list.txt | cut -d'.' -f1 | sort | uniq -c
```

Chuyển đổi định dạng dữ liệu:

```
sed 's/::/,/g' input.txt > output.txt
```

8. Xử lý file đầu vào và đầu ra

Đếm số dòng của tập tin:

```
wc -l file.txt
```

Đếm số từ của tập tin:

```
wc -w file.txt
```

Tìm dòng dài nhất trong tập tin:

```
awk '{ if (length > max) { max = length; line = $0 } } END { print  
line }' file.txt
```

Tìm kiếm một chuỗi trong tập tin:

```
grep "chuoi" file.txt
```

Phần nâng cao

1. Xử lý tham số dòng lệnh

Kiểm tra số lượng tham số:

```
if [ $# -eq 0 ]; then
    echo "Chua nhap tham so!"
    exit 1
fi
```

Lặp qua tất cả tham số:

```
for arg in "$@"; do
    echo "Tham so: $arg"
done
```

Xử lý shift để dịch tham số:

```
while [ $# -gt 0 ]; do
    echo "Tham so: $1"
    shift
done
```

Xử lý tham số cụ thể:

```
while [ $# -gt 0 ]; do
    case "$1" in
        -c) echo "Gap -c, tham so tiep theo la: $2"; shift ;;
        *) echo "Tham so khac: $1" ;;
    esac
    shift
done
```

2. Xử lý chuỗi & tập tin

Tách chuỗi theo dấu phân cách:

```
echo "Nguyen Van A::LTMT2::K8::8::9::8" | tr ":@" ","
```

Cắt chuỗi bằng cut:

```
echo "ngochan@yahoo.com.vn" | cut -d'@' -f1 # Lay ten tai khoan
echo "ngochan@yahoo.com.vn" | cut -d'@' -f2 | cut -d'.' -f1 # Lay nha
cung cap
```

Tìm chuỗi trong tập tin (thay grep):

```
while IFS= read -r line; do
    case "$line" in
        *"$tukhoa"*) echo "Tim thay: $line" ;;
    esac
done < file.txt
```

Đổi đuôi file (.jpg → .png):

```
for file in *.jpg; do
    mv "$file" "${file%.jpg}.png"
done
```

Lấy dòng cụ thể từ tập tin:

```
sed -n '5p' file.txt # In dong thu 5
```

Đếm số dòng chứa chuỗi:

```
grep -c "tu_khoa" file.txt
```

Tìm kiếm file trong thư mục:

```
find . -name "*chuoi_tim_kiem"
```

Tìm dòng dài nhất trong file:

```
awk '{ if (length > max) { max = length; line = $0 } } END { print  
line }' file.txt
```

3. Xử lý số & toán học

Cộng tổng các tham số:

```
sum=0  
for num in "$@"; do  
    sum=$((sum + num))  
done  
echo "Tong cac so la: $sum"
```

Tìm số lớn nhất trong danh sách:

```
max=$1  
for num in "$@"; do  
    if [ $num -gt $max ]; then  
        max=$num  
    fi  
done  
echo "So lon nhat la: $max"
```

Sinh số ngẫu nhiên (1–10):

```
random=$((RANDOM % 10 + 1))  
echo "So ngau nhien: $random"
```

Tính giai thừa với for:

```
fact=1  
for (( i=1; i<=$1; i++ )); do  
    fact=$((fact * i))  
done  
echo "Giai thua cua $1 la $fact"
```

Tính số ngày trong tháng:

```
cal $1 $2 | awk 'NF {DAYS = $NF}; END {print DAYS}'
```

4. Điều kiện & vòng lặp

Kiểm tra file tồn tại:

```
if [ -f "file.txt" ]; then  
    echo "File ton tai"  
fi
```

Kiểm tra quyền file:

```
if [ -r "file.txt" ]; then echo "Co quyen doc"; fi
if [ -w "file.txt" ]; then echo "Co quyen ghi"; fi
if [ -x "file.txt" ]; then echo "Co quyen thuc thi"; fi
```

Vòng lặp while đọc file:

```
while IFS= read -r line; do
    echo "$line"
done < file.txt
```

Trò chơi đoán số:

```
random=$((RANDOM % 10 + 1))
attempts=0
while [ $attempts -lt 5 ]; do
    read -p "Nhap so doan (1-10): " guess
    if [ "$guess" -eq "$random" ]; then
        echo "Ban da doan dung!"
        exit 0
    fi
    attempts=$((attempts + 1))
done
echo "Ban da het luot!"
```

5. Quản lý hệ thống & tiến trình

Xem danh sách tiến trình:

```
ps aux | grep process_name
```

Dừng tiến trình:

```
kill <PID>
```

Lưu & khôi phục PATH:

```
echo $PATH > oldpath.txt
export PATH=""
source oldpath.txt
```

Thay đổi dấu nhắc lệnh (PS1):

```
export PS1="MyShell> "
```

Thay đổi PS2:

```
export PS2="Tiep tục nhập> "
```

6. Xử lý email & dữ liệu có cấu trúc

Tách email thành các phần:

```
email="ngochan@yahoo.com.vn"
user=$(echo $email | cut -d'@' -f1)
domain=$(echo $email | cut -d'@' -f2 | cut -d'.' -f1)
ext=$(echo $email | cut -d'@' -f2 | cut -d'.' -f2-)
echo "User: $user, Domain: $domain, Extension: $ext"
```


Thống kê số lượng email theo nhà cung cấp:

```
cut -d'@' -f2 email_list.txt | cut -d'.' -f1 | sort | uniq -c
```

Chuyển đổi định dạng dữ liệu (:: → ,):

```
sed 's/::/,/g' input.txt > output.txt
```

7. Xử lý file đầu vào và đầu ra

Đếm số dòng của tập tin:

```
wc -l file.txt
```

Đếm số từ của tập tin:

```
wc -w file.txt
```

Tìm kiếm chuỗi trong file (không dùng grep):

```
while read line; do
    if [[ "$line" == *"$1"* ]]; then
        echo "Tim thay: $line"
    fi
done < "$2"
```

Cắt tách tập tin

1. cut – Cắt chuỗi theo dấu phân cách

Lệnh cut giúp cắt một phần của chuỗi hoặc dữ liệu theo **cột** hoặc **khoảng ký tự**.

Cắt theo dấu phân cách (-d)

Ví dụ 1: Cắt họ tên từ danh sách học sinh

```
echo "Nguyen Van A,20.03.1997,8,9,10" | cut -d',' -f1
```

Kết quả: Nguyen Van A

Giải thích:

- -d',': Dấu phẩy (,) là dấu phân cách.
- -f1: Lấy cột thứ 1 (họ tên).

Ví dụ 2: Lấy năm sinh từ danh sách học sinh

```
echo "Nguyen Van A,20.03.1997,8,9,10" | cut -d',' -f2 | cut -d'.' -f3
```

Kết quả: 1997

Giải thích:

- cut -d',' -f2: Lấy phần **ngày sinh**.
- cut -d'.' -f3: Lấy **năm sinh** từ định dạng dd.mm.yyyy.

Cắt theo số ký tự (-c)

Ví dụ 3: Lấy 5 ký tự đầu tiên của chuỗi

```
echo "Nguyen Van A" | cut -c1-5
```

Kết quả: Nguye

2. awk – Cắt chuỗi theo cột & xử lý dữ liệu nâng cao

Lệnh awk giúp cắt chuỗi theo khoảng trắng hoặc dấu phân cách tùy chỉnh, đồng thời có thể tính toán hoặc lọc dữ liệu.

Cắt theo khoảng trắng

Ví dụ 4: Lấy cột thứ 2 trong danh sách

```
echo "Nguyen Van A 8 9 10" | awk '{print $2}'
```

Kết quả: Van

Giải thích:

- print \$2: In cột thứ 2.

Ví dụ 5: Lấy cột cuối cùng (NF – Number of Fields)

```
echo "Nguyen Van A 8 9 10" | awk '{print $NF}'
```

Kết quả: 10

Giải thích:

- \$NF: Cột cuối cùng của dòng.

Cắt theo dấu phân cách (-F)

Ví dụ 6: Lấy điểm Tin học từ danh sách CSV

```
echo "Nguyen Van A,,20.03.1997,,8,,9,,10" | awk -F",," '{print $5}'
```

Kết quả: 10

Giải thích:

- -F",,": Định dạng phân cách là ,,.
- {print \$5}: In cột thứ 5 (điểm Tin học).

Lọc và tính toán dữ liệu

Ví dụ 7: Tính tổng điểm các môn

```
echo "Nguyen Van A,,20.03.1997,,8,,9,,10" | awk -F",," '{print $1, "
Tong diem:", $5 + $6 + $7}'
```

Kết quả: Nguyen Van A Tong diem: 27

3. head – Lấy N dòng đầu tiên của tập tin

Lệnh head giúp lấy một số dòng đầu tiên từ tập tin.

Ví dụ 8: Hiển thị 5 học sinh đầu tiên

```
head -5 students.csv
```

Kết quả:

```
Ho va ten,,Ngay sinh,,Toan,,Tieng Anh,,Tin hoc
Le Trong Hung,,17.03.2002,,10,,9,,10
Hoang Tien Dung,,01.02.1998,,10,,6,,10
Nguyen Thanh Dat,,20.03.1997,,8,,4,,5
Luong Van Duc,,20.02.2002,,10,,4,,8
```

Ví dụ 9: Lấy 3 dòng đầu tiên của danh sách

```
head -n 3 students.csv
```

Kết quả:

```
Ho va ten,,Ngay sinh,,Toan,,Tieng Anh,,Tin hoc
Le Trong Hung,,17.03.2002,,10,,9,,10
Hoang Tien Dung,,01.02.1998,,10,,6,,10
```

4. tail – Lấy N dòng cuối cùng của tập tin

Lệnh tail giúp lấy một số dòng cuối từ tập tin.

Ví dụ 10: Hiển thị 5 học sinh cuối cùng

```
tail -5 students.csv
```

Kết quả:

```
Bui Cong Hieu,,23.05.2002,,4,,9,,6
Le Van Hieu,,01.01.2003,,7,,5,,9
Nguyen Trung Hieu,,06.07.2002,,7,,4,,4
Nguyen Huy Hoang,,30.11.1999,,5,,4,,5
```

Ví dụ 11: Lấy 2 dòng cuối cùng

```
tail -n 2 students.csv
```

Kết quả:

```
Nguyen Trung Hieu,,06.07.2002,,7,,4,,4
Nguyen Huy Hoang,,30.11.1999,,5,,4,,5
```

Ví dụ 12: Theo dõi file log.txt trong thời gian thực

```
tail -f log.txt
```

Ứng dụng:

- Theo dõi syslog, access.log, error.log của hệ thống.

Tổng kết

Lệnh	Mô tả	Ví dụ
<code>cut -d',' -fN</code>	Cắt cột thứ N theo dấu phân cách	<code>cut -d',' -f1 file.csv</code>
<code>awk '{print \$N}'</code>	Lấy cột N từ dữ liệu	<code>awk '{print \$3}' file.txt</code>
<code>awk -F',' '{print \$N}'</code>	Cắt theo dấu ,	<code>awk -F',' '{print \$2}' file.csv</code>
<code>head -N file.txt</code>	Lấy N dòng đầu	<code>head -5 file.txt</code>
<code>tail -N file.txt</code>	Lấy N dòng cuối	<code>tail -3 file.txt</code>
<code>tail -f file.log</code>	Theo dõi file log thời gian thực	<code>tail -f /var/log/syslog</code>

Sắp xếp

1. Sắp xếp dữ liệu theo thứ tự mặc định

Ví dụ 1: Sắp xếp danh sách họ tên

```
echo -e "Nguyen Van A\nLe Thi B\nHoang Minh C" | sort
```

Kết quả:

```
Hoang Minh C
Le Thi B
Nguyen Van A
```

Giải thích:

- `sort` sắp xếp theo thứ tự bảng chữ cái (ASCII).

2. Sắp xếp số (-n)

Ví dụ 2: Sắp xếp điểm số

```
echo -e "5\n10\n2\n8" | sort -n
```

Kết quả:

```
2
5
8
10
```

Giải thích:

- `-n`: Sắp xếp theo số thay vì theo ASCII.
- Nếu không có `-n`, nó sẽ hiểu `10 < 2` (vì 1 đứng trước 2 trong ASCII).

3. Sắp xếp theo thứ tự giảm dần (-r)

Ví dụ 3: Sắp xếp số theo thứ tự giảm dần

```
echo -e "5\n10\n2\n8" | sort -nr
```

Kết quả:

10
8
5
2

Giải thích:

- -r: Đảo ngược thứ tự (giảm dần).

4. Sắp xếp theo cột cụ thể (-k)

Ví dụ 4: Sắp xếp danh sách học sinh theo điểm Tin học

File students.csv:

```
Ho va ten,,Ngay sinh,,Toan,,Tieng Anh,,Tin hoc
Le Trong Hung,,17.03.2002,,10,,9,,10
Hoang Tien Dung,,01.02.1998,,10,,6,,10
Nguyen Thanh Dat,,20.03.1997,,8,,4,,5
Luong Van Duc,,20.02.2002,,10,,4,,8
```

Lấy cột điểm Tin học (cột 7) để sắp xếp:

```
sort -t",," -k7 -nr students.csv
```

Kết quả:

```
Le Trong Hung,,17.03.2002,,10,,9,,10
Hoang Tien Dung,,01.02.1998,,10,,6,,10
Luong Van Duc,,20.02.2002,,10,,4,,8
Nguyen Thanh Dat,,20.03.1997,,8,,4,,5
```

Giải thích:

- -t",,": Đặt dấu ,, làm dấu phân cách cột.
- -k7: Sắp xếp theo cột 7 (điểm Tin học).
- -n: Sắp xếp theo số.
- -r: Sắp xếp giảm dần.

5. Sắp xếp file nhưng giữ nguyên dòng đầu (-h, -stable)

Ví dụ 5: Sắp xếp danh sách có tiêu đề

Nếu file có dòng đầu là tiêu đề, ta cần bỏ qua dòng đầu tiên khi sắp xếp:

```
( head -n 1 students.csv && tail -n +2 students.csv | sort -t",," -k7
-nr ) > sorted_students.csv
```

Giải thích:

- head -n 1 students.csv: Lấy dòng đầu tiên (tiêu đề).
- tail -n +2 students.csv: Lấy tất cả dòng trừ dòng đầu tiên.
- sort ...: Sắp xếp phần còn lại.
- > sorted_students.csv: Ghi kết quả ra file mới.

6. Sắp xếp không phân biệt chữ hoa/thường (-f)

Ví dụ 6: Sắp xếp danh sách tên không phân biệt chữ cái viết hoa

```
echo -e "apple\nBanana\norange\nAPPLE" | sort -f
```

Kết quả:

```
apple
APPLE
Banana
orange
```

Giải thích:

- -f: Không phân biệt chữ hoa và chữ thường.

7. Loại bỏ dòng trùng lặp (-u)

Ví dụ 7: Xóa dòng trùng lặp sau khi sắp xếp

```
echo -e "apple\nBanana\napple\norange" | sort -u
```

Kết quả:

```
Banana
apple
orange
```

Giải thích:

- -u: Giữ lại các dòng duy nhất.

8. Sắp xếp theo định dạng nhân văn (-h)

Ví dụ 8: Sắp xếp file có kích thước (GB, MB, KB)

File sizes.txt:

```
5K
100M
2G
10K
500M
```

Sắp xếp theo dung lượng thực tế:

```
sort -h sizes.txt
```

Kết quả:

```
5K
10K
100M
500M
2G
```

Giải thích:

- -h: Sắp xếp theo kích thước thực (K, M, G).

9. Sắp xếp theo định dạng ngày tháng

Ví dụ 9: Sắp xếp danh sách theo ngày sinh

File `dob.txt`:

```
17-03-2002
01-02-1998
20-03-1997
24-03-1999
```

Sắp xếp theo thời gian (tăng dần):

```
sort -t '-' -k3,3n -k2,2n -k1,1n dob.txt
```

Kết quả:

```
20-03-1997
01-02-1998
24-03-1999
17-03-2002
```

Giải thích:

- `-t '-'`: Đặt dấu `-` làm dấu phân cách.
- `-k3,3n`: Sắp xếp theo cột năm.
- `-k2,2n`: Sắp xếp theo cột tháng.
- `-k1,1n`: Sắp xếp theo cột ngày.

10. Kết hợp sort với uniq để đếm số lần xuất hiện

Ví dụ 10: Đếm số lần xuất hiện của mỗi từ

```
echo -e "apple\nbanana\napple\norange\nbanana\nbanana" | sort | uniq -c
```

Kết quả:

```
2 apple
3 banana
1 orange
```

Giải thích:

- `sort`: Sắp xếp dữ liệu trước khi đếm.
- `uniq -c`: Đếm số lần xuất hiện của từng dòng.

Tổng kết

Lệnh	Chức năng	Ví dụ
sort	Sắp xếp tăng dần (mặc định)	sort file.txt
sort -r	Sắp xếp giảm dần	sort -r file.txt
sort -n	Sắp xếp số	sort -n file.txt
sort -kN	Sắp xếp theo cột N	sort -k3 file.txt
sort -t" " -kN	Sắp xếp theo cột với dấu phân cách	sort -t"," -k2 file.csv
sort -u	Loại bỏ dòng trùng	sort -u file.txt
sort -h	Sắp xếp theo đơn vị (K, M, G)	sort -h file.txt
sort -f	Không phân biệt hoa/thường	sort -f file.txt

Shuf

shuf là một lệnh trong Linux dùng để **xáo trộn** các dòng của một tệp hoặc đầu vào, sau đó in ra kết quả. Nó thường được sử dụng để chọn ngẫu nhiên một hoặc nhiều dòng từ danh sách.

Cú pháp cơ bản:

```
shuf [tùy chọn] [tệp]
```

Một số ví dụ:

1. Xáo trộn nội dung của một tệp:

```
shuf file.txt
```

→ In ra các dòng của file.txt theo thứ tự ngẫu nhiên.

2. Chọn 5 dòng ngẫu nhiên từ một tệp:

```
shuf -n 5 file.txt
```

→ Chọn ngẫu nhiên 5 dòng từ file.txt.

3. Tạo số ngẫu nhiên trong khoảng 1–100:

```
shuf -i 1-100 -n 5
```

→ In ra 5 số ngẫu nhiên từ 1 đến 100.

4. Xáo trộn danh sách file trong thư mục /etc:

```
ls /etc | shuf -n 10
```

→ Lấy ngẫu nhiên 10 file/thư mục từ /etc.

Bạn có thể dùng shuf khi cần chọn ngẫu nhiên dòng trong file hoặc danh sách bất kỳ.

Toán tử xử lý chuỗi trong Shell

Toán tử	Mô tả	Kết quả
\${info#*,}	Xóa đến dấu , đầu tiên	"Ngọc Ánh,,20-12-22,,3,,4"
\${info##*,}	Xóa đến dấu , cuối cùng	"4"
\${info%*,}	Xóa từ dấu , cuối cùng về sau	"Nguyễn Thị Ngọc Ánh,,20-12-22,,3,,"
\${info%%*,}	Xóa từ dấu , đầu tiên về sau	"Nguyễn Thị Ngọc Ánh"

3. Thay thế chuỗi

Toán tử	Chức năng
<code>\${var/pattern/string}</code>	Thay phần khớp đầu tiên của pattern bằng string
<code>\${var//pattern/string}</code>	Thay tất cả phần khớp của pattern bằng string

Ví dụ:

```
text="apple banana apple orange"
echo "${text/apple/grape}"      # Thay "apple" đầu tiên thành "grape"
echo "${text//apple/grape}"    # Thay tất cả "apple" thành "grape"
```

- `${text/apple/grape}`: "grape banana apple orange"
- `${text//apple/grape}`: "grape banana grape orange"

Tóm tắt:

- `#` & `##`: Xóa phần khớp ở **đầu chuỗi** (ngắn nhất / dài nhất).
- `%` & `%%`: Xóa phần khớp ở **cuối chuỗi** (ngắn nhất / dài nhất).
- `/` & `//`: Thay thế phần khớp trong chuỗi (chỉ lần đầu / tất cả).

Read

Tùy chọn	Mô tả
<code>-p</code>	Hiển thị lời nhắc (prompt) trước khi nhập
<code>-r</code>	Không cho phép xử lý ký tự <code>\</code> (backslash) đặc biệt
<code>-s</code>	Ẩn đầu vào (thường dùng khi nhập mật khẩu)
<code>-t</code>	Thiết lập thời gian chờ khi nhập dữ liệu
<code>-n</code>	Chỉ đọc số ký tự nhất định rồi tự động dừng
<code>-d</code>	Đặt ký tự kết thúc nhập (EOF) thay vì Enter
<code>-a</code>	Lưu dữ liệu nhập vào một mảng
<code>-u</code>	Đọc từ file descriptor cụ thể (thay vì stdin)

1. `-p` – Hiển thị prompt

Dùng để hiển thị nội dung trước khi nhập:

```
read -p "Nhập tên của bạn: " name
echo "Xin chào, $name!"
```

⇒ Xuất hiện: Nhập tên của bạn: _ (người dùng nhập vào)

2. `-r` – Không xử lý backslash (`\`)

Mặc định, `read` xử lý `\` như một ký tự đặc biệt. Dùng `-r` để vô hiệu hóa điều này.

```
read -r line
echo "Bạn đã nhập: $line"
```

⇒ Nếu nhập `Hello \ World`, kết quả vẫn giữ nguyên `Hello \ World`.

3. -s – Nhập dữ liệu ẩn (ví dụ: mật khẩu)

```
read -s -p "Nhap mat khau: " password
echo -e "\nBan da nhap mat khau (duoc an)."
```

⇒ Khi nhập, nội dung không hiển thị trên màn hình.

4. -t – Đặt thời gian chờ khi nhập

Nếu người dùng không nhập trong thời gian quy định, lệnh `read` sẽ tự động dừng.

```
read -t 5 -p "Nhap ten trong 5 giay: " name
echo "Ten cua ban: $name"
```

⇒ Nếu sau 5 giây không nhập, chương trình tiếp tục mà không lưu dữ liệu.

5. -n – Giới hạn số ký tự nhập

Dùng để giới hạn số ký tự cần nhập trước khi `read` tự động dừng.

```
read -n 4 -p "Nhap ma PIN (4 chu so): " pin
echo -e "\nMa PIN cua ban la: $pin"
```

⇒ Sau khi nhập 4 ký tự, `read` tự động kết thúc mà không cần nhấn Enter.

6. -d – Chọn ký tự kết thúc nhập

Mặc định, `read` dừng khi nhấn Enter. Nhưng có thể thay đổi ký tự kết thúc bằng `-d`.

```
read -d ";" -p "Nhap chuoai (ket thuc bang dau cham phay): " input
echo "Chuoai ban nhap: $input"
```

⇒ Nhập Hello; thì chương trình dừng ngay khi gặp ;.

7. -a – Lưu dữ liệu vào mảng

```
read -a arr
echo "Phan tu dau tien: ${arr[0]}"
echo "Phan tu thu hai: ${arr[1]}"
```

⇒ Nhập Python Bash Linux, mảng chứa ["Python", "Bash", "Linux"].

8. -u – Đọc từ file descriptor

Lệnh `read` mặc định đọc từ stdin, nhưng với `-u`, bạn có thể đọc từ file descriptor khác.

```
exec 3< myfile.txt # Mo file duoi dang descriptor 3
read -u 3 line
echo "Dong dau tien cua file: $line"
exec 3<&- # Dong file descriptor
```

uniq

```
uniq file.txt
```

- Chỉ giữ lại một dòng duy nhất nếu có các dòng liên tiếp trùng nhau.
- **Lưu ý:** `uniq` chỉ loại bỏ dòng trùng liên tiếp. Nếu các dòng giống nhau không đứng cạnh nhau, `uniq` không loại bỏ chúng.

Ví dụ:

Giả sử tệp `names.txt` có nội dung:

```
Alice
Alice
Bob
Bob
Bob
Charlie
Alice
```

Khi chạy:

```
uniq names.txt
```

Kết quả:

```
Alice
Bob
Charlie
Alice
```

⇒ Alice cuối cùng không bị xóa vì không đứng cạnh một dòng Alice khác.

Một số tùy chọn quan trọng của `uniq`

1. `-c` – Đếm số lần xuất hiện của mỗi dòng

Hiển thị số lần mỗi dòng xuất hiện (chỉ với các dòng trùng nhau liên tiếp).

```
uniq -c names.txt
```

Kết quả:

```
2 Alice
3 Bob
1 Charlie
1 Alice
```

2. `-d` – Chỉ hiển thị các dòng bị trùng

```
uniq -d names.txt
```

Kết quả:

```
Alice
Bob
```

⇒ Chỉ hiển thị các dòng có ít nhất **một lần trùng** (bị lặp lại liên tiếp).

3. -u – Chỉ hiển thị các dòng không bị trùng

```
uniq -u names.txt
```

Kết quả:

Charlie
Alice

⇒ Chỉ giữ lại các dòng xuất hiện đúng **một lần** trong tệp.

4. -i – Bỏ qua phân biệt chữ hoa chữ thường

```
uniq -i names.txt
```

Nếu names.txt có:

Alice
alice
BOB
Bob
bob
Charlie

Kết quả:

Alice
BOB
Charlie

⇒ Alice và alice được coi là giống nhau, BOB, Bob, bob cũng vậy.

5. -f N – Bỏ qua N cột đầu tiên trước khi kiểm tra trùng

001 Alice
002 Alice
003 Bob
004 Bob
005 Charlie

Chạy:

```
uniq -f 1 data.txt
```

Kết quả:

001 Alice
003 Bob
005 Charlie

⇒ `uniq` bỏ qua cột số đầu tiên khi kiểm tra trùng.

6. -s N – Bỏ qua N ký tự đầu tiên trên mỗi dòng

Giả sử data.txt có:

```
ID1 Alice
ID2 Alice
ID3 Bob
ID4 Bob
ID5 Charlie
```

Chạy:

```
uniq -s 4 data.txt
```

Kết quả:

```
ID1 Alice
ID3 Bob
ID5 Charlie
```

⇒ uniq bỏ qua 4 ký tự đầu tiên (IDx) khi kiểm tra trùng.

Tổng kết

Lệnh	Chức năng
uniq file.txt	Xóa dòng trùng nhau liên tiếp
uniq -c file.txt	Hiển thị số lần xuất hiện của từng dòng
uniq -d file.txt	Chỉ hiển thị các dòng bị trùng
uniq -u file.txt	Chỉ hiển thị các dòng không bị trùng
uniq -i file.txt	Không phân biệt chữ hoa, chữ thường
uniq -f N file.txt	Bỏ qua N cột đầu tiên khi so sánh
uniq -s N file.txt	Bỏ qua N ký tự đầu tiên khi so sánh